Lanzamiento Explícito de Excepciones

Uso de la palabra reservada `throw`

Área de Ciencias de la Computación

Universidad Autónoma de San Luis Potosí
Facultad de Ingeniería

Tecnologías Orientadas a Objetos

Noviembre 2026

Repaso: Programación Defensiva y Estados del Objeto

- En sesiones anteriores definimos la **Programación Defensiva** como el enfoque donde asumimos que todo dato externo es sospechoso.

Repaso: Programación Defensiva y Estados del Objeto

- En sesiones anteriores definimos la **Programación Defensiva** como el enfoque donde asumimos que todo dato externo es sospechoso.
- En el paradigma Orientado a Objetos, proteger el estado interno de un objeto (Encapsulamiento) es fundamental.

Repaso: Programación Defensiva y Estados del Objeto

- En sesiones anteriores definimos la **Programación Defensiva** como el enfoque donde asumimos que todo dato externo es sospechoso.
- En el paradigma Orientado a Objetos, proteger el estado interno de un objeto (Encapsulamiento) es fundamental.
- **El problema:** ¿Qué sucede cuando un método recibe parámetros que violan la lógica de negocio, o el objeto se encuentra en un estado donde no puede realizar la operación solicitada?

Repaso: Programación Defensiva y Estados del Objeto

- En sesiones anteriores definimos la **Programación Defensiva** como el enfoque donde asumimos que todo dato externo es sospechoso.
- En el paradigma Orientado a Objetos, proteger el estado interno de un objeto (Encapsulamiento) es fundamental.
- **El problema:** ¿Qué sucede cuando un método recibe parámetros que violan la lógica de negocio, o el objeto se encuentra en un estado donde no puede realizar la operación solicitada?
- **La solución:** Debemos rechazar la operación *inmediatamente* antes de corromper los datos.

¿Qué es `throw` en Java?

La palabra reservada `throw` se utiliza para **lanzar (detonar) explícitamente** una excepción desde cualquier bloque de código.

Características principales:

- Transforma un simple error lógico en un evento formal del sistema.
- Interrumpe el flujo normal de ejecución instantáneamente.
- Obliga a la JVM a buscar un bloque `catch` o terminar el programa.
- Requiere instanciar un objeto que herede de `Throwable` (ej. `new Exception()`).

Para entenderlo de forma sencilla: El Freno de Emergencia

Imagina que tu método es un tren avanzando por las vías (las líneas de código).

- Usar `throw` es literalmente **jalar el freno de emergencia**.
- Si detectas que algo no cuadra (por ejemplo, te piden retirar dinero pero el saldo es cero), usas `throw` para detener el tren *antes* de que ocurra un desastre.
- En el momento que Java lee la palabra `throw`, **cancela todo lo que sigue** en ese método. Ni una sola línea más de ese bloque se ejecuta.

La Diferencia Fundamental: `throw` vs `throws`

Es común confundirlos, pero tienen propósitos distintos:

- **`throw`:** Es la **acción**. Detona físicamente el error en el código. Va dentro de las llaves `{}` del método.
- **`throws`:** Es la **advertencia**. Va en la firma del método. Le avisa a los demás qué errores podrían salir de ahí.

Analogía: *`throws` es la etiqueta del menú que dice "Este platillo puede contener maní" (advertencia). `throw` es el momento exacto en que alguien sufre la reacción alérgica (la acción/emergencia).*

Ejemplo 1: Uso exclusivo de `throw`

Se usa solo `throw` para validaciones rápidas (Excepciones No Comprobadas). No es obligatorio usar `throws` aquí.

```java
public class Validador {
    public void registrarEdad(int edad) {
        if (edad < 18) {
            // ACCION: Jalamos el freno de emergencia aqui mismo.
            throw new IllegalArgumentException("El usuario debe ser mayor.");
        }
        // Si la edad es menor a 18, el programa JAMAS llega a esta linea.
        System.out.println("Edad registrada correctamente.");
    }
}
```

Ejemplo 2: Uso combinado (`throw` y `throws`)

Para errores más serios (Excepciones Comprobadas como `IOException`), Java te obliga a usar ambas cosas.

```java
import java.io.IOException;

public class SistemaArchivos {
    // ADVERTENCIA (throws): Avisamos que este metodo es peligroso
    public void validarRuta(String ruta) throws IOException {
        if (ruta == null || ruta.isEmpty()) {
            // ACCION (throw): Ocurre el error fisico
            throw new IOException("La ruta del archivo esta vacia.");
        }
        System.out.println("Ruta valida, procediendo a leer...");
    }
}
```

Ejemplo 3: Uso exclusivo de `throws`

A veces tu método no tiene ningún `throw`, pero llama a un método que sí lo tiene. Usas `throws` para delegar el problema (lavarte las manos).

```java
import java.io.IOException;

public class GestorDocumentos {
    // No hay "throw" adentro, pero pasamos el problema hacia arriba
    public void procesarDocumento() throws IOException {
        SistemaArchivos sistema = new SistemaArchivos();

        // Si falla aqui, el error sube y cancela este metodo automaticamente.
        sistema.validarRuta("");

        System.out.println("Proceso finalizado."); // Nunca se ejecuta si hay error
    }
}
```

Resumen: `throw` vs `throws`

`throw` **(Lanzar)**	`throws` **(Declarar)**
Va **dentro** del método (`{}`).	Va en la **firma** del método.
Crea la excepción (`new Exception()`).	Solo nombra las clases de excepción.
Detiene el flujo inmediatamente.	Advierte a quien llame al método.

¿En qué momentos específicos usar `throw`?

1. **Validación de parámetros de entrada (Fail-Fast)**
   - Úsalo inmediatamente al inicio de tus métodos (constructores o setters) para validar los argumentos.
   - *Ejemplo:* Si un método calcula el área y recibe un radio negativo, lanza una `IllegalArgumentException`.
2. **Protección del estado del objeto**
   - Cuando el objeto no puede completar una operación por su estado actual.
   - *Ejemplo:* Intentar hacer `pop()` en una Pila (Stack) que ya está vacía (`IllegalStateException`).

¿En qué momentos específicos usar `throw`? (Cont.)

3. Restricciones lógicas de negocio

- Cuando se rompe una regla específica del dominio del problema.
- *Ejemplo:* En un sistema escolar, un alumno no puede inscribir materias si tiene un adeudo en biblioteca.

4. Manejo de fallos externos no recuperables

- Si tu clase lee un archivo, pero el archivo está corrupto. Tu clase no sabe cómo avisarle al usuario final, así que lanzas una excepción para que la interfaz gráfica la atrape y muestre una alerta.

Ejemplo Comparativo: El Setter

Mal diseño (Silencia el error, permite estado inconsistente):

```java
public void setEdad(int edad) {
    if (edad >= 0) {
        this.edad = edad;
    }
    // Si es negativo, no hace nada, el usuario no se entera del error.
}
```

Buen diseño Orientado a Objetos (Programación a la Defensiva):

```java
public void setEdad(int edad) {
    if (edad < 0) {
        // Detenemos la ejecucion y avisamos del uso incorrecto
        throw new IllegalArgumentException("La edad no puede ser negativa: " + edad)
            ;
    }
    this.edad = edad;
}
```

Ejercicio Práctico: Sistema Bancario (Parte 1)

Objetivo: Proteger el estado aplicando el principio de *Fail-Fast* mediante el lanzamiento de excepciones (`throw`).

Clase `CuentaBancaria`: Atributos `titular`, `saldo`, `activa` (true por defecto).

- **Constructor:** Validar que `titular` no esté vacío o nulo (lanza `IllegalArgumentException`) y que `saldoInicial` no sea negativo (lanza `IllegalArgumentException`).
- **`depositar(double monto)`:**
  - Si la cuenta no está activa, lanza `IllegalStateException`.
  - Si `monto` ≤ 0, lanza `IllegalArgumentException` ("Monto debe ser mayor a cero").
  - Si es válido, sumar al saldo.

Ejercicio Práctico: Sistema Bancario (Parte 2)

Continuación de los métodos de **`CuentaBancaria`**:

- **`retirar(double monto)`:**
  - Si no está activa o `monto` ≤ 0, lanza las excepciones correspondientes.
  - Si `monto` $>$ `saldo`, lanza `IllegalStateException` ("Fondos insuficientes").
  - Si es válido, restar del saldo.
- **`transferir(CuentaBancaria destino, double monto)`:**
  - Si `destino` es nula o es la misma cuenta, lanza `IllegalArgumentException`.
  - **Flujo:** Llama internamente a `this.retirar(monto)` y luego a `destino.depositar(monto)`.
- **`bloquearCuenta()` y `desbloquearCuenta()`:**
  - Cambian el atributo `activa` a `false` o `true`.

Ejercicio Práctico: Pruebas de Estrés (`Main`)

Crea la clase principal `Main`. Cada una de las siguientes acciones debe estar envuelta en su propio bloque `try-catch` (imprimiendo con `System.err.println`):

1. **Prueba de creación:** Intentar crear cuenta con titular `""`.
2. **Prueba de saldo:** Intentar crear cuenta con saldo de `-500`.
3. **Prueba de retiro sin fondos:** Retirar $5,000 de una cuenta que tiene $1,000.
4. **Prueba de cuenta bloqueada:** Bloquear una cuenta válida y luego intentar depositarle $500.
5. **Prueba de transferencia:** Transferir exitosamente de "María" a "Carlos". (Nota cómo al fallar un retiro, el depósito en la otra cuenta nunca ocurre, protegiendo el sistema).

¿Preguntas?

Pasemos a la codificación del ejercicio en su IDE.